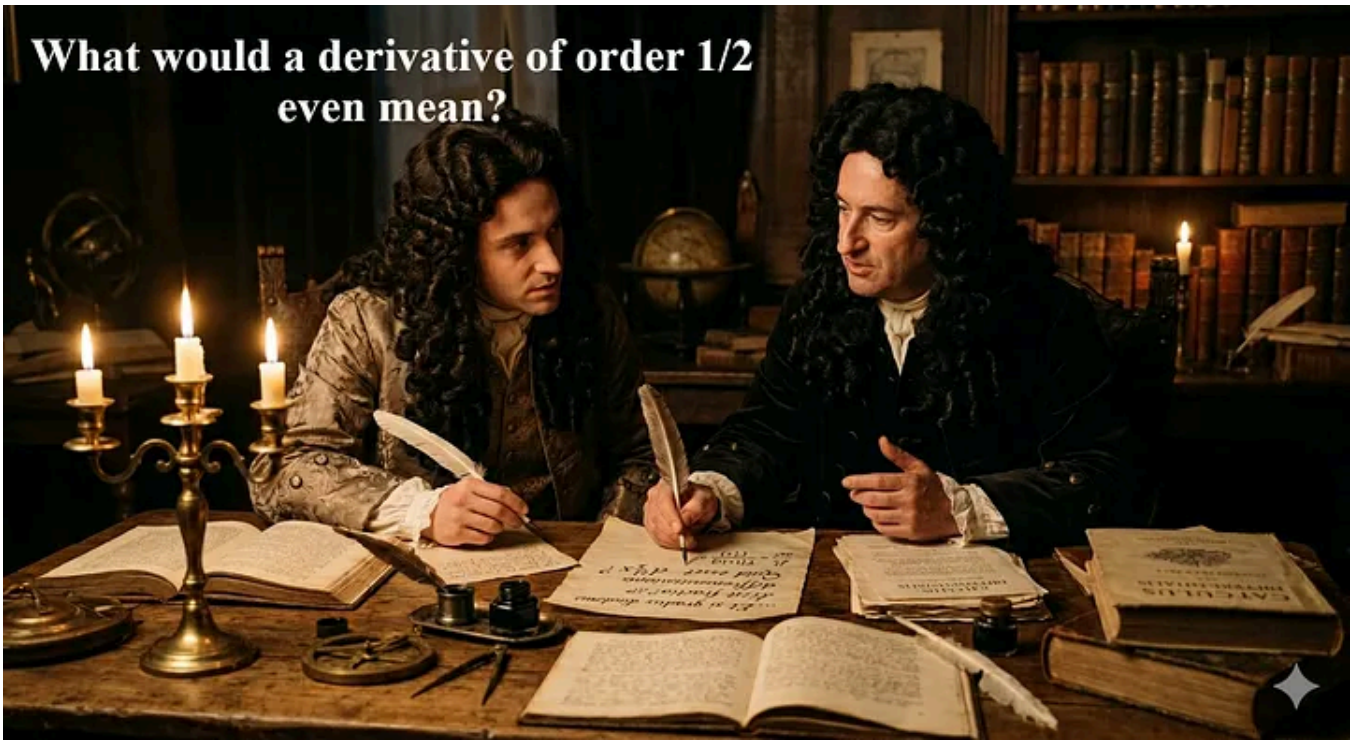
How Fractional Calculus Entered Finance:

15 min read · 2 days ago

Simon Leung

A Short Introduction to the Hurst Exponent and Fractal R/S Analysis



Gemini_Generated_Image

The Leibniz–L'Hôpital Exchange: The Spark That Ignited Fractional Calculus

The origins of fractional calculus can be traced to a brief but remarkably forward-looking exchange between *Gottfried Wilhelm Leibniz* and *Guillaume de l'Hôpital* in 1695. At the time, differential calculus was still in its infancy, and mathematicians were actively exploring the boundaries of this new symbolic language. In one of their letters, Leibniz casually introduced the idea that

differentiation need not be restricted to whole numbers. L'Hôpital, intrigued by the conceptual elasticity of Leibniz's notation, pressed him with a now-famous question: *What would a derivative of order 1/2 even mean?*

Leibniz's response was both playful and prophetic. He admitted that a half-order derivative seemed paradoxical under the mathematical framework of the day, yet he insisted that the idea was neither meaningless nor absurd. Instead, he suggested that such a notion would "one day lead to useful consequences," anticipating — more than two centuries early — the eventual development of a rigorous theory of derivatives and integrals of arbitrary order.

This short correspondence, almost an intellectual curiosity at the time, planted the conceptual seed for what would become fractional calculus. Later mathematicians such as Liouville, Riemann, Grünwald, Letnikov, and Caputo would formalize the definitions that Leibniz could only speculate about. But the philosophical leap — the willingness to extend differentiation beyond the integers — belongs to this moment, when two of the era's sharpest minds entertained a question that seemed impossible yet irresistibly rich.

Further readings:
What is Fractional Calculus?
What the Heck is a Half Derivative?
Fractional Calculus Demystified: [Part 1] [Part 2] [Part 3]

Picture Egypt in the early 1900s.

The Nile — ancient, life-giving, unpredictable — was both a blessing and a threat. Floods could nourish the land or destroy entire harvests. The British-Egyptian authorities wanted to build long-term water reservoirs, but to do that, they needed to understand the river's rhythms over centuries, not just seasons.

So they turned to a quiet, meticulous hydrologist: **Harold Edwin Hurst.**

He wasn't a mathematician. He wasn't a physicist. He was a man obsessed with water — and with understanding how nature remembers.

Hurst spent **62 years** studying the Nile. He dug through ancient records, measured river levels, rainfall, silt, and flood cycles. He compiled **thousands of years** of data — a dataset so large and so old that modern statisticians still marvel at it.

But something bothered him.

When he applied the standard statistical tools of the time — tools built on the assumption of randomness and short memory — the Nile refused to fit.

The river *remembered*.

Hurst used what we now call **rescaled range analysis (R/S)**. He expected the relationship to scale like:

$$R/S \sim N^{0.5}$$

because that's what classical Brownian motion predicts.
But the Nile said otherwise.
The exponent wasn't 0.5. ***It was closer to 0.72–0.75***.

This meant:
• Floods and droughts were not independent
• The river had long-term persistence
• Nature carried a memory far longer than anyone expected
Hurst didn't know it yet, but he had discovered a crack in the foundation of classical randomness.

. . .

The Hurst exponent **H** determines *how much memory* a process has. Fractional calculus provides the *mathematical machinery* to model that memory using **fractional integrals and derivatives**.

. . .

From the Nile to Fractional Brownian Motion

In the early 1960s, *Benoit Mandelbrot* was wandering through the scientific world like a man searching for a hidden language. He was fascinated by roughness —

coastlines, cotton prices, turbulence, noise — anything that refused to fit the smooth curves of classical mathematics.

Mandelbrot realized that Hurst had uncovered something profound: a natural process that was ***self-similar***, ***persistent***, and ***fractal-like*** long before the word "fractal" existed.

In 1968, Mandelbrot and Van Ness built a mathematical model that could produce exactly the kind of behavior Hurst saw: **fractional Brownian motion (fBM)** — a generalization of Brownian motion with a tunable memory parameter ***H***.

Fractional Brownian motion is literally constructed using fractional integrals.

The FBM $B_H(t)$ is defined as:

$$B_H(t) = \frac{1}{\Gamma(H+\frac{1}{2})} \int_{-\infty}^{t} \left[(t-s)^{H-\frac{1}{2}} - (-s)^{H-\frac{1}{2}} \right] dB(s)$$

where $B(s)$ is standard Brownian motion.

from my previous article

Main Differences between Bm and fBm

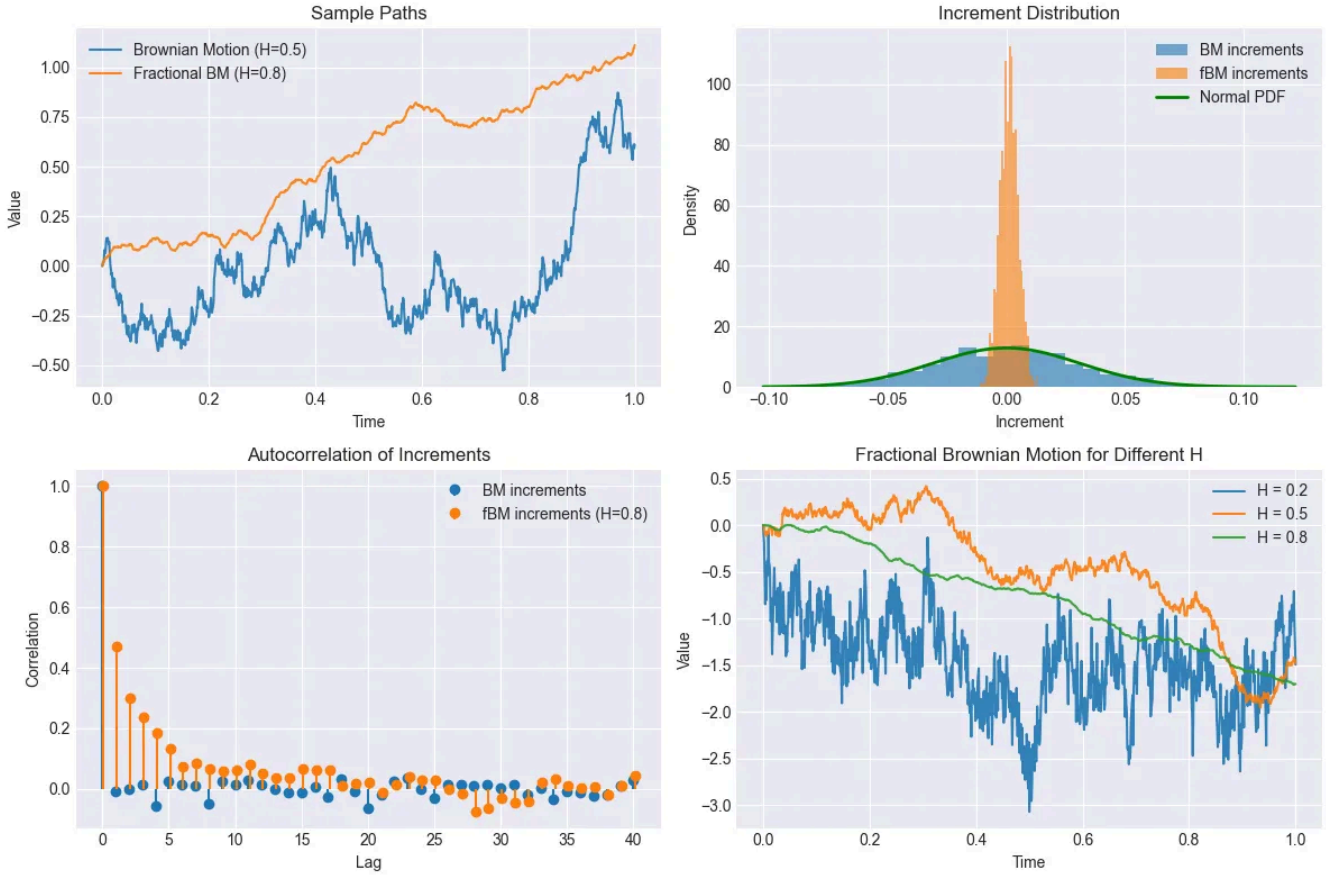
Property	Brownian Motion	Fractional Brownian Motion
Hurst exponent	H = 0.5 only	0 < H < 1
Increment dependence	Independent	Correlated
Memory	No memory	Long memory
Markov property	Yes	No
Semi-martingale	Yes	No (except H = 0.5)
Variance growth	$Var(W_t) = t$	$Var(B_H(t)) = t^{2H}$
Self-similarity	H = 0.5	Any H
Ito calculus	Applicable	Not directly applicable

Simulate the behavior of Brownian motion (BM) and fractional Brownian motion (fBM)

```
import numpy as np
import matplotlib.pyplot as plt
from fbm import FBM          # pip install fbm
from statsmodels.tsa.stattools import acf  # pip install statsmodels

# ------------------------------
# 1. Parameters
# ------------------------------
T = 1.0          # total time
N = 1000         # number of steps
dt = T / N
t = np.linspace(0, T, N + 1)

np.random.seed(42)

# ------------------------------
# 2. Simulate Brownian Motion (H = 0.5)
# ------------------------------
# Standard BM increments ~ N(0, dt)
bm_increments = np.sqrt(dt) * np.random.randn(N)
```

```
bm_path = np.concatenate([[0], np.cumsum(bm_increments)])

# ----------------------------
# 3. Simulate fractional Brownian Motion (H = 0.8)
# ----------------------------
H_fbm = 0.8
fbm_gen = FBM(n=N, hurst=H_fbm, length=T, method='daviesharte')
fbm_path = fbm_gen.fbm()  # length N+1

# fBM increments
fbm_increments = np.diff(fbm_path)

# ----------------------------
# 4. Increment distributions (histograms)
# ----------------------------
# For a fair comparison, use same number of increments
bm_increments_for_hist = bm_increments
fbm_increments_for_hist = fbm_increments

# ----------------------------
# 5. Autocorrelation of increments
# ----------------------------
max_lag = 40

bm_acf = acf(bm_increments_for_hist, nlags=max_lag, fft=True)
fbm_acf = acf(fbm_increments_for_hist, nlags=max_lag, fft=True)
lags = np.arange(max_lag + 1)

# ----------------------------
# 6. fBM for different H values
# ----------------------------
H_values = [0.2, 0.5, 0.8]
fbm_paths_H = {}

for H in H_values:
    fbm_gen_H = FBM(n=N, hurst=H, length=T, method='daviesharte')
    fbm_paths_H[H] = fbm_gen_H.fbm()

# ----------------------------
# 7. Plotting
# ----------------------------
plt.style.use('seaborn-v0_8-darkgrid')
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
ax1, ax2, ax3, ax4 = axes.ravel()

# ---- Top-left: Sample Paths (BM vs fBM H=0.8) ----
ax1.plot(t, bm_path, label='Brownian Motion (H=0.5)', color='tab:blue', alpha=0.9)
ax1.plot(t, fbm_path, label='Fractional BM (H=0.8)', color='tab:orange', alpha=0.9)
ax1.set_title('Sample Paths')
ax1.set_xlabel('Time')
ax1.set_ylabel('Value')
ax1.legend(loc='best')
```

```
# ---- Top-right: Increment Distribution ----
bins = 30
ax2.hist(bm_increments_for_hist, bins=bins, density=True, alpha=0.6,
         label='BM increments', color='tab:blue')
ax2.hist(fbm_increments_for_hist, bins=bins, density=True, alpha=0.6,
         label='fBM increments', color='tab:orange')

# Overlay normal density for reference
from scipy.stats import norm
x_vals = np.linspace(
    min(bm_increments_for_hist.min(), fbm_increments_for_hist.min()),
    max(bm_increments_for_hist.max(), fbm_increments_for_hist.max()),
    200
)
mu = 0.0
sigma = np.std(bm_increments_for_hist)  # use BM std as reference
ax2.plot(x_vals, norm.pdf(x_vals, mu, sigma), color='green', lw=2, label='Normal PD

ax2.set_title('Increment Distribution')
ax2.set_xlabel('Increment')
ax2.set_ylabel('Density')
ax2.legend(loc='best')

# ---- Bottom-left: Autocorrelation of Increments ----
ax3.stem(lags, bm_acf, linefmt='tab:blue', markerfmt='o', basefmt=' ', label='BM in
ax3.stem(lags + 0.1, fbm_acf, linefmt='tab:orange', markerfmt='o', basefmt=' ', lab

ax3.set_title('Autocorrelation of Increments')
ax3.set_xlabel('Lag')
ax3.set_ylabel('Correlation')
ax3.legend(loc='best')

# ---- Bottom-right: fBM for Different H ----
colors = {0.2: 'tab:blue', 0.5: 'tab:orange', 0.8: 'tab:green'}
for H in H_values:
    ax4.plot(t, fbm_paths_H[H], label=f'H = {H}', color=colors[H], alpha=0.9)

ax4.set_title('Fractional Brownian Motion for Different H')
ax4.set_xlabel('Time')
ax4.set_ylabel('Value')
ax4.legend(loc='best')

plt.tight_layout()
plt.show()
```



1. Sample Paths (Top-Left Plot)

What you're seeing: Two time-series curves generated over the same time grid:

- **Blue:** Standard Brownian Motion (H = 0.5)
- **Orange:** Fractional Brownian Motion (H = 0.8)

Interpretation:

- **BM (H = 0.5)** has *no memory*. Its increments are independent, so the path is jagged and highly irregular.
- **fBM with H = 0.8** exhibits **persistence**:
  . Positive increments tend to be followed by positive increments.
  . Negative increments tend to be followed by negative increments.
  . This produces a smoother, more trending path.

Why it matters: This plot visually demonstrates the core effect of the Hurst exponent:

- **H = 0.5 → classical random walk**
- **H > 0.5 → long-memory, smoother, trending behavior**

2. Increment Distribution (Top-Right Plot)

What you're seeing: Histograms of the increments (differences between consecutive points) for:

- BM increments (blue)
- fBM increments (orange)
- A green curve showing a fitted normal distribution

Interpretation:

- Both BM and fBM increments are **approximately Gaussian**, which is expected because both processes are Gaussian processes.
- The **centered bell shape** indicates mean ≈ 0.
- The **spread** (variance) may differ slightly depending on how the fBM was generated.

Key insight: Even though fBM has memory, its *marginal increment distribution* is still Gaussian. The difference between BM and fBM is not in the distribution of increments — it's in the **correlation structure** of those increments.

3. Autocorrelation of Increments (Bottom-Left Plot)

What you're seeing: Autocorrelation functions (ACF) of increments for:

- BM (blue)
- fBM with H = 0.8 (orange)

Interpretation:

- **BM increments:**
  . ACF fluctuates around zero.
  . This confirms independent increments — the defining property of Brownian motion.
- **fBM increments (H = 0.8):**
  . ACF starts positive and decays slowly.
  . This indicates long-range dependence or persistence.
  . The higher the H, the slower the decay.

Why it matters: This is the *statistical fingerprint* of fractional Brownian motion. Even though the increments are Gaussian, they are **not independent** when $H \neq 0.5$.

4. Fractional Brownian Motion for Different H (Bottom-Right Plot)

What you're seeing: Three sample paths of fBM with different Hurst exponents:

- H = 0.2 (blue)
- H = 0.5 (orange)
- H = 0.8 (green)

Interpretation:

- **H = 0.2 (anti-persistent):**
  . Very jagged, rough path
  . Increments tend to reverse direction
  . Negative correlation between increments
- **H = 0.5 (standard BM):**
  . Classical random walk behavior
  . No memory
- **H = 0.8 (persistent):**
  . Smooth, trending path
  . Strong positive correlation between increments

Why it matters: This plot shows how the controls the *roughness* of the process:

H Value	Behavior	Memory	Roughness
H < 0.5	Anti-persistent	Negative	Very rough
H = 0.5	Brownian	None	Rough
H > 0.5	Persistent	Positive	Smooth

. . .

Run *Rescaled Range (R/S) Analysis* and *Detrended Fluctuation Analysis* for S&P500

> *Detrended Fluctuation Analysis (DFA) is a technique for measuring the same power law scaling observed through R/S Analysis. It was introduced specifically to address nonstationaries. [1]*

DFA computes the scaling of detrended fluctuations:

$$F(n) \sim n^{\alpha}$$

> *The scaling exponent α is a generalization of the Hurst exponent, with the precise value giving information about the series self-correlations: [2]*
> *α<1/2: anti-correlated*
> *α≃1/2: uncorrelated, white noise*
> *α>1/2: correlated*
> *α≃1: 1/f-noise, pink noise*
> *α>1: non-stationary, unbounded*
> *α≃3/2: Brownian noise*

H = α (for stationary series where 0 < α < 1)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
from scipy.stats import linregress

# ============================================================
# Download SP500
# ============================================================

# Load data
print("Loading S&P 500 data...")

ticker = "^GSPC"
data = yf.download(ticker, start="2000-01-01", progress=False)
data.columns = data.columns.get_level_values(0)
data.columns = [''.join(col).strip() for col in data.columns.values]
close = data["Close"].dropna()

# Log returns
r = np.log(close / close.shift(1)).dropna().values

N = len(r)
```

```
print(f"Observations: {len(close)} prices, {len(r)} returns")
print(f"Period: {data.index[0].date()} to {data.index[-1].date()}")



# ============================================================
# Common scales used by BOTH methods
# ============================================================

MIN_SCALE = 16
MAX_SCALE = N // 12

scales = np.unique(
    np.round(
        np.logspace(
            np.log10(MIN_SCALE),
            np.log10(MAX_SCALE),
            20
        )
    ).astype(int)
)

print(f"Scale range: {MIN_SCALE} <= n <= {MAX_SCALE}")
print(f"Number of unique log-spaced scales = {len(scales)}")
print(f"Scale list: {scales}")
print("\n\n")

def rs_analysis(x,
                scales,
                min_windows=8):
    """
    Rescaled Range (R/S) Analysis to estimate the Hurst exponent H.

    Parameters
    ----------
    x : 1-D array-like
        Input time series (e.g., log-returns).
    scales : array-like
        Window sizes (scales) to evaluate.
    min_windows : int, default 8
        Minimum number of non-overlapping windows required for a scale
        to be included in the regression.

    Returns
    -------
    dict
        scales      : array of valid scales used
        RSvalues    : mean R/S for each scale
        H           : Hurst exponent (slope of log(R/S) vs log(scale))
        R2          : coefficient of determination of the fit
        intercept   : intercept of the regression line
```

```
    Interpretation of H (Hurst exponent):
    -------------------------------------
    H < 0.5  → Anti-persistent (mean-reverting) process
    H = 0.5  → Brownian motion / random walk (uncorrelated)
    H > 0.5  → Persistent (trend-following) process
    """

    N = len(x)
    rs_values = []
    valid_scales = []
    for n in scales:
        num_windows = N // n
        if num_windows < min_windows:
            continue
        rs_window = []
        for i in range(num_windows):
            segment = x[i*n:(i+1)*n]
            mean_seg = np.mean(segment)
            y = np.cumsum(segment - mean_seg)
            R = np.max(y) - np.min(y)
            S = np.std(segment, ddof=1)
            if S > 0:
                rs_window.append(R / S)

        if rs_window:
            valid_scales.append(n)
            rs_values.append(np.mean(rs_window))

    valid_scales = np.array(valid_scales)
    rs_values = np.array(rs_values)

    x_reg = np.log10(valid_scales)
    y_reg = np.log10(rs_values)

    slope, intercept, r_value, _, _ = linregress(
        x_reg,
        y_reg
    )

    return {
        "scales": valid_scales,
        "RSvalues": rs_values,
        "H": slope,
        "R2": r_value**2,
        "intercept": intercept
    }

def dfa(x,
        scales,
        min_windows=8):
    """
    Detrended Fluctuation Analysis (DFA) to estimate the scaling exponent α.
```

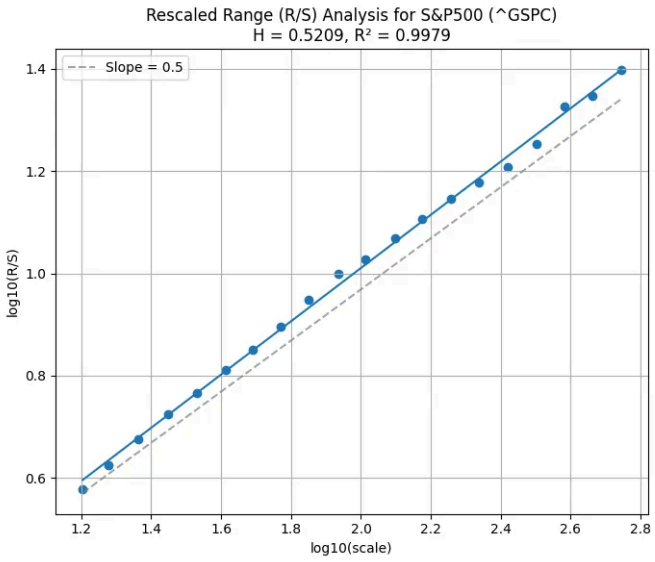
```
    Parameters
    ----------
    x : 1-D array-like
        Input time series (e.g., log-returns).
    scales : array-like
        Window sizes (scales) to evaluate.
    min_windows : int, default 8
        Minimum number of non-overlapping windows required for a scale
        to be included in the regression.

    Returns
    -------
    dict
        scales      : array of valid scales used
        Fvalues     : mean fluctuation for each scale
        alpha       : scaling exponent (slope of log(F) vs log(scale))
        R2          : coefficient of determination of the fit
        intercept   : intercept of the regression line

    Interpretation of α (DFA scaling exponent):
    --------------------------------------------
    α < 0.5  → Anti-correlated (mean-reverting) process
    α = 0.5  → Uncorrelated white noise
    0.5 < α < 1.0 → Positively correlated (persistent) process
    α = 1.0  → 1/f noise (pink noise)
    α = 1.5  → Brownian noise (integrated white noise)
    α > 1.5  → Non-stationary / strongly correlated process
    """

    N = len(x)
    profile = np.cumsum(x - np.mean(x))

    F_values = []
    valid_scales = []

    for n in scales:
        num_windows = N // n
        if num_windows < min_windows:
            continue
        flucts = []
        for i in range(num_windows):
            segment = profile[i*n:(i+1)*n]
            t = np.arange(n)
            coeffs = np.polyfit(t, segment, 1)
            trend = np.polyval(coeffs, t)
            rms = np.sqrt(
                np.mean(
                    (segment - trend)**2
                )
            )

            flucts.append(rms)
```

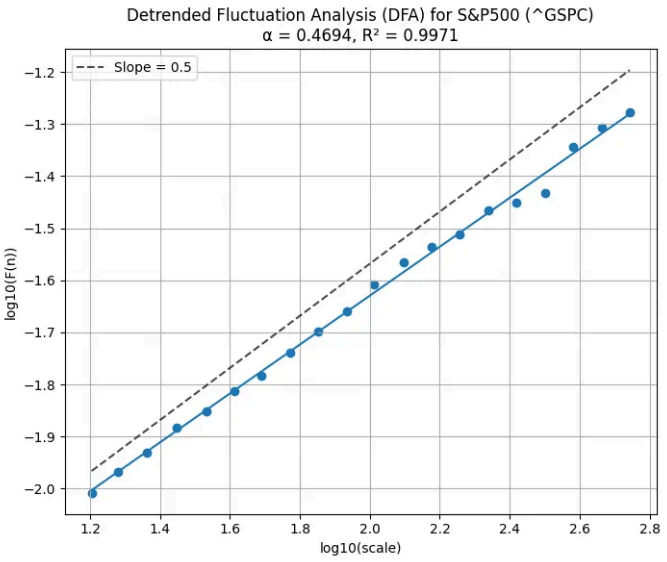
```
            if flucts:
                valid_scales.append(n)
                F_values.append(np.mean(flucts))

        valid_scales = np.array(valid_scales)
        F_values = np.array(F_values)

        x_reg = np.log10(valid_scales)
        y_reg = np.log10(F_values)

        slope, intercept, r_value, _, _ = linregress(
            x_reg,
            y_reg
        )

        return {
            "scales": valid_scales,
            "Fvalues": F_values,
            "alpha": slope,
            "R2": r_value**2,
            "intercept": intercept
        }

# ------------------------------------------------------------
#  Interpretation helpers
# ------------------------------------------------------------
def interpret_H_and_alpha(H, alpha):
    """
    Combined interpretation of Hurst exponent (R/S) and DFA alpha.

    Classification thresholds (based on standard theory):
    ------------------------------------------------------
    H (Hurst exponent):
        H < 0.5  → Anti-persistent (mean-reverting)
        H = 0.5  → Brownian motion (random walk)
        H > 0.5  → Persistent (trend-following)

    α (DFA scaling exponent):
        α < 0.5  → Anti-correlated (mean-reverting)
        α = 0.5  → Uncorrelated white noise
        α > 0.5  → Positively correlated (persistent)
        α = 1.5  → Brownian noise (integrated white noise)
    """

    # Handle NaN cases
    if np.isnan(H) and np.isnan(alpha):
        return "Both H and α are NaN: insufficient data or numerical issue."
    if np.isnan(H):
        return "H is NaN; α interpretation only.\n" + interpret_DFA_alpha(alpha)
    if np.isnan(alpha):
        return "α is NaN; H interpretation only.\n" + interpret_statistical_Hurst(H

    # Classify H using the standard thresholds: H < 0.5 anti-persistent, H = 0.5 Br
```

```
    if H > 0.5:
        H_regime = "persistent"
    elif H < 0.5:
        H_regime = "anti-persistent"
    else:
        H_regime = "Brownian (random walk)"

    # Classify α using the standard thresholds: α < 0.5 anti-correlated, α = 0.5 wh
    if alpha > 0.5:
        A_regime = "persistent (positively correlated)"
    elif alpha < 0.5:
        A_regime = "anti-correlated (anti-persistent)"
    else:
        A_regime = "uncorrelated white noise"

    # Combined logic
    if H_regime == A_regime.split(" ")[0]:
        combined = f"Both estimators agree: {H_regime} behavior."
    else:
        combined = (
            "Mixed signals: R/S and DFA disagree.\n"
            "- This usually indicates weak long-memory\n"
            "- Or non-stationarity / volatility clustering\n"
            "- Or estimator sensitivity to scale choices"
        )

    # Build message
    msg = (
        f"[Combined H + α Interpretation]\n"
        f"H ≈ {H:.4f} → {H_regime}\n"
        f"α ≈ {alpha:.4f} → {A_regime}\n"
        f"\n{combined}\n"
        f"\nInterpretation guide (Hurst exponent H):\n"
        f"  H < 0.5  → Anti-persistent (mean-reverting)\n"
        f"  H = 0.5  → Brownian motion (random walk)\n"
        f"  H > 0.5  → Persistent (trend-following)\n"
        f"\nInterpretation guide (DFA scaling exponent α):\n"
        f"  α < 0.5  → Anti-correlated (mean-reverting)\n"
        f"  α = 0.5  → Uncorrelated white noise\n"
        f"  α > 0.5  → Positively correlated (persistent)\n"
        f"  α = 1.5  → Brownian noise (integrated white noise)\n"
    )

    return msg

# ============================================================
# Run analyses
# ============================================================
rs_result = rs_analysis(
    r,
    scales,
    min_windows=8
)
```

```
dfa_result = dfa(
    r,
    scales,
    min_windows=8
)

# Create a single figure with 1 row and 2 columns
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# ============================================================
# Plot 1: R/S Analysis (Left Subplot)
# ============================================================
ax1 = axes[0]
x_rs = np.log10(rs_result["scales"])
y_rs = np.log10(rs_result["RSvalues"])

# Reference line with slope = 0.5 (using its own intercept to align with data)
y_ref_rs = rs_result["intercept"] + 0.5 * x_rs
ax1.plot(x_rs, y_ref_rs, color="gray", linestyle="--", alpha=0.7, label="Slope = 0.

ax1.scatter(x_rs, y_rs)
ax1.plot(x_rs, rs_result["intercept"] + rs_result["H"] * x_rs)
ax1.set_xlabel("log10(scale)")
ax1.set_ylabel("log10(R/S)")
ax1.set_title(
    f"Rescaled Range (R/S) Analysis for S&P500 (^GSPC)\n" f"H = {rs_result['H']:.4f
)
ax1.grid(True)
ax1.legend()

# ============================================================
# Plot 2: DFA (Right Subplot)
# ============================================================
ax2 = axes[1]
x_dfa = np.log10(dfa_result["scales"])
y_dfa = np.log10(dfa_result["Fvalues"])

# Reference line with slope = 0.5 (using its own intercept to align with data)
y_ref_dfa = dfa_result["intercept"] + 0.5 * x_dfa
ax2.plot(x_dfa, y_ref_dfa, color="black", linestyle="--", alpha=0.7, label="Slope =

ax2.scatter(x_dfa, y_dfa)
ax2.plot(x_dfa, dfa_result["intercept"] + dfa_result["alpha"] * x_dfa)
ax2.set_xlabel("log10(scale)")
ax2.set_ylabel("log10(F(n))")
ax2.set_title(
    f"Detrended Fluctuation Analysis (DFA) for S&P500 (^GSPC)\n" f"α = {dfa_result[
)
ax2.grid(True)
ax2.legend()

# Adjust layout and display
```

```
plt.tight_layout()
plt.savefig("./Documents/RS-DFA.png")
plt.show()

# ============================================================
# Summary
# ============================================================

print("\n==============================")
print("R/S Analysis for S&P500 (^GSPC)")
print("==============================")
print(f"Hurst H  = {rs_result['H']:.4f}")
print(f"R²       = {rs_result['R2']:.4f}")

print("\n==============================")
print("DFA for S&P500 (^GSPC)")
print("==============================")
print(f"Alpha α  = {dfa_result['alpha']:.4f}")
print(f"R²       = {dfa_result['R2']:.4f}")

print("\n==============================")
print("Relationship")
print("==============================")
print("For stationary log-return data:")
print("    H ≈ α")
print(f"Difference = {abs(rs_result['H'] - dfa_result['alpha']):.4f}")

print("\n==============================")
print("Interpretation")
print("==============================")
print(interpret_H_and_alpha(rs_result['H'], dfa_result['alpha']))
```





```
Loading S&P 500 data...
Observations: 6659 prices, 6658 returns
Period: 2000-01-03 to 2026-06-26
Scale range: 16 <= n <= 554
Number of unique log-spaced scales = 20
Scale list: [ 16  19  23  28  34  41  49  59  71  86 103 125 150 181 218 263 317 38
 460 554]

==============================
R/S Analysis for S&P500 (^GSPC)
==============================
Hurst H  = 0.5209
R²       = 0.9979

==============================
DFA for S&P500 (^GSPC)
==============================
Alpha α  = 0.4694
R²       = 0.9971

==============================
Relationship
==============================
For stationary log-return data:
    H ≈ α
Difference = 0.0515

==============================
Interpretation
==============================
[Combined H + α Interpretation]
H ≈ 0.5209 → persistent
α ≈ 0.4694 → anti-correlated (anti-persistent)

Mixed signals: R/S and DFA disagree.
- This usually indicates weak long-memory
- Or non-stationarity / volatility clustering
- Or estimator sensitivity to scale choices

Interpretation guide (Hurst exponent H):
  H < 0.5  → Anti-persistent (mean-reverting)
  H = 0.5  → Brownian motion (random walk)
  H > 0.5  → Persistent (trend-following)

Interpretation guide (DFA scaling exponent α):
  α < 0.5  → Anti-correlated (mean-reverting)
  α = 0.5  → Uncorrelated white noise
  α > 0.5  → Positively correlated (persistent)
  α = 1.5  → Brownian noise (integrated white noise)
```

. . .

Fractal (R/S) Analysis employs the same mathematical procedure as Rescaled Range (R/S) Analysis; however, it places greater emphasis on the fractal-scaling interpretation.

The fractal dimension D is a **measure of roughness** or **irregularity** of a price path.

Fractal Dimension: D = 2 − H

Range	Interpretation	Market Regime
D = 1.0	H = 1.0	Perfectly smooth, deterministic trend (theoretical limit)
1.0 < D < 1.5	0.5 < H < 1.0	Persistent / trending markets (low roughness)
D = 1.5	H = 0.5	Random walk (benchmark, maximum entropy)
1.5 < D < 2.0	0 < H < 0.5	Anti-persistent / mean-reverting markets (high roughness)
D = 2.0	H = 0	Maximum roughness, perfectly oscillating (theoretical limit)

Practical bounds for financial data:

- Typical range: **D ∈ [1.3, 1.7]** or roughly **D ∈ [1.2, 1.8]**
- Extreme crises may briefly push D > 1.8 (very choppy)
- Strong bull trends may push D < 1.3 (very smooth)

Fractional order d is the **degree of long-memory** in the series.

Fractional Order: d = H − 0.5

Range	Interpretation	Memory Type
d = +0.5	H = 1.0	Maximum long-memory (theoretical limit, perfect persistence)
0 < d < 0.5	0.5 < H < 1.0	Long-memory, positive autocorrelation (momentum)
d = 0	H = 0.5	No memory (standard Brownian motion)
−0.5 < d < 0	0 < H < 0.5	Anti-persistent, negative autocorrelation (mean-reversion)
d = −0.5	H = 0	Maximum anti-persistence (theoretical limit)

Practical bounds for financial data:

- Typical range: **d ∈ [−0.2, +0.2]** or roughly **d ∈ [−0.3, +0.3]**
- Strong trending periods: d up to ~+0.15
- Crisis/volatile periods: d down to ~−0.15
- Very rough volatility models: d ≈ −0.4 (H ≈ 0.1)

H	D	d	Regime	Roughness
1	1	0.5	**Perfect trend** (deterministic line)	Smooth
0.7	1.3	0.2	**Persistent trending** (strong positive autocorrelation)	Moderately smooth
0.6	1.4	0.1	**Weak persistence** (trend but noisy)	Slightly smooth
0.5	1.5	0	**Random walk** (Brownian motion, uncorrelated increments)	Neutral
0.4	1.6	−0.1	**Weak mean-reversion** (anti-persistent)	Slightly rough
0.3	1.7	−0.2	**Strong mean-reversion** (oscillatory)	Moderately rough
0.1	1.9	−0.4	**Extreme anti-persistence** (very rough, negatively correlated)	Very rough
0	2	−0.5	**Perfect anti-persistence** (maximally negatively correlated increments)	Maximally rough

. . .

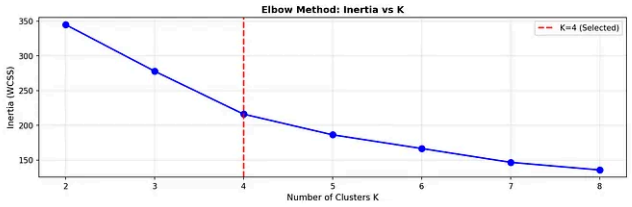
Build S&P 500 Fractal Rolling Hurst Regime Detection

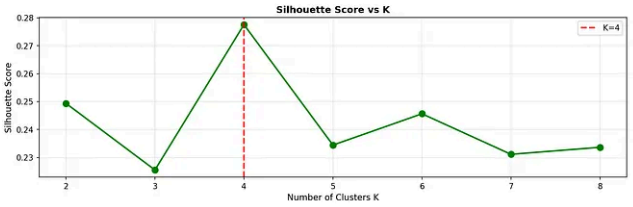
[Full Python code is here]

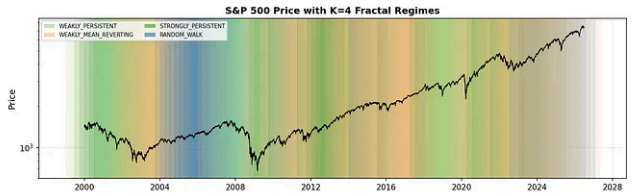
```
Loading S&P 500 data...
N = 6658 returns | 2000-01-04 to 2026-06-25
Prices: 6659 points | Dates: 6659 points
```

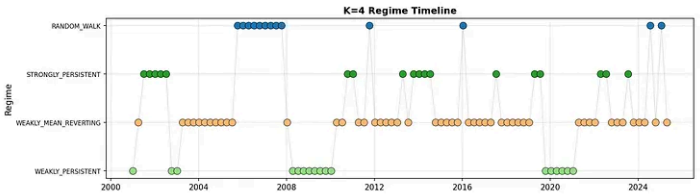
```
Fitting FractalRegimeClassifier with auto K-detection...


Descriptive statistics
            H_rs  D_higuchi     d_frac  alpha_dfa  volatility
count  98.000000  98.000000  98.000000  98.000000   98.000000
mean    0.575881   1.043562   0.075881   0.480429   18.127336
std     0.044773   0.047080   0.044773   0.060783    6.679187
min     0.454943   0.942283  -0.045057   0.306971    9.971126
25%     0.554953   1.014477   0.054953   0.447810   13.097684
50%     0.574878   1.039713   0.074878   0.479101   16.339479
75%     0.603079   1.070281   0.103079   0.519547   21.706131
max     0.671877   1.227613   0.171877   0.679244   35.250376


======================================================================
AUTO-DETECTING OPTIMAL K (Elbow + Silhouette)
======================================================================
  K      Inertia   Silhouette     Calinski Davies-Bouldin
  2       345.06       0.2494        40.32         1.3803
  3       278.02       0.2255        36.22         1.3713
  4       216.16       0.2776        39.69         1.1279
  5       186.40       0.2344        37.87         1.2390
  6       166.69       0.2456        35.69         1.1081
  7       146.76       0.2311        35.47         1.0840
  8       135.79       0.2336        33.54         1.0933

Silhouette recommends: K = 4 (score = 0.2776)
Elbow method suggests: K = 4
Consensus: K = 4 (both methods agree)
======================================================================

======================================================================
REGIME SUMMARY
======================================================================
               regime  count       pct   H_mean    H_std   D_mean   d_mean  alpha_m
    WEAKLY_PERSISTENT     17 17.346939 0.581659 0.031391 0.992103 0.081659    0.492
WEAKLY_MEAN_REVERTING     50 51.020408 0.572033 0.019844 1.051551 0.072033    0.475
  STRONGLY_PERSISTENT     18 18.367347 0.635105 0.023906 1.067915 0.135105    0.548
          RANDOM_WALK     13 13.265306 0.501121 0.032362 1.046406 0.001121    0.388

======================================================================
CLUSTER NAMES: {1: 'RANDOM_WALK', 3: 'WEAKLY_PERSISTENT', 0: 'STRONGLY_PERSISTENT',
======================================================================

Creating 16-panel dashboard...
```

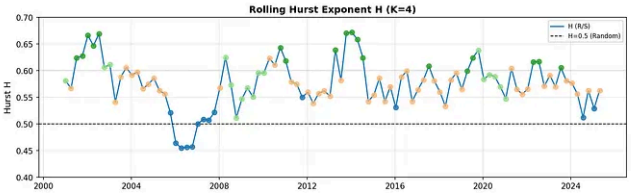
S&P 500 Fractal Rolling Hurst Regime Detection Pipeline (K=4)
H (Hurst) | D (Fractal Dimension) | d (Fractional Order) | α (DFA)
Auto K-Detection (Elbow + Silhouette) | 2-Year Window | 3-Month Step

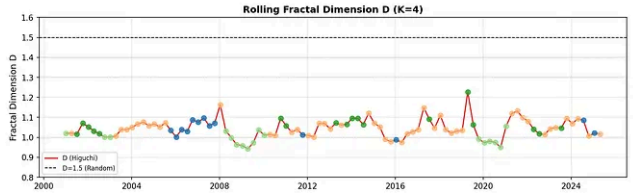


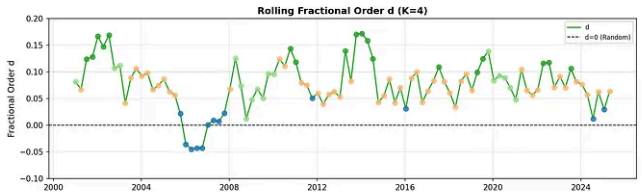


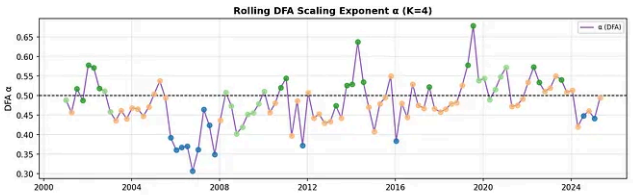


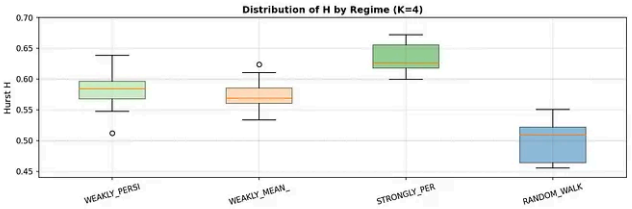


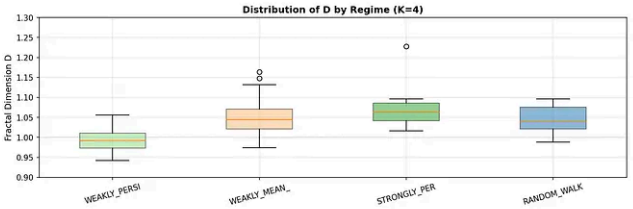


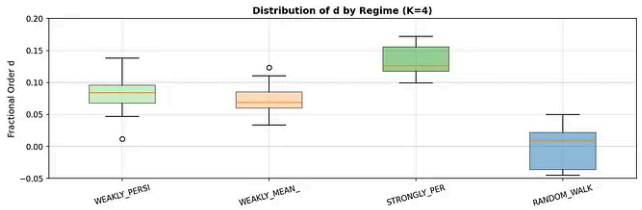


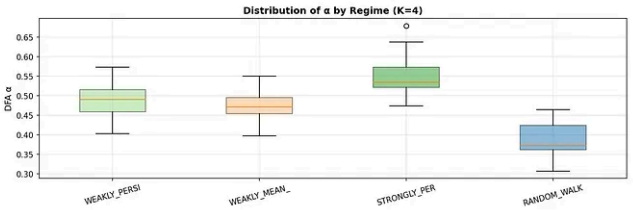


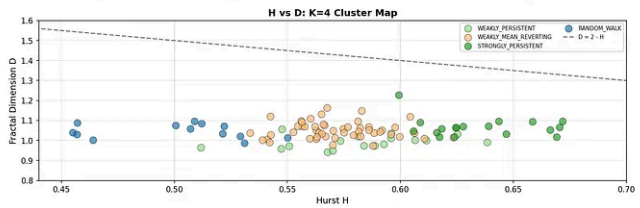


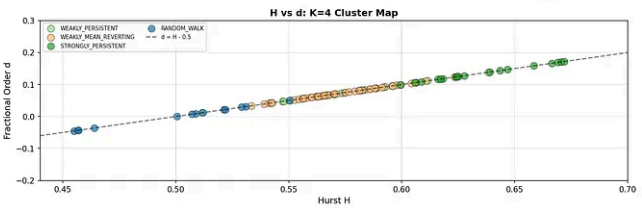


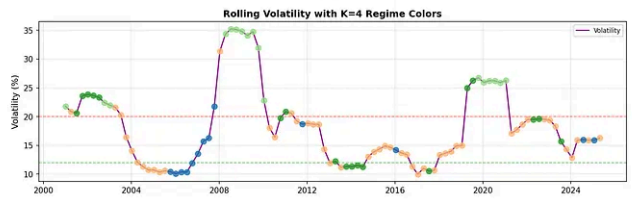


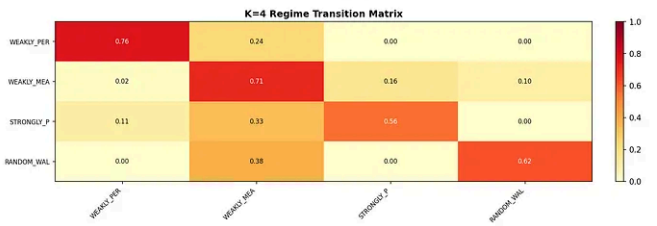












• • •

Afterword:

[1] Readers may explore alternative window lengths (scale ranges) for both R/S and DFA, similar to the approaches used in the following three articles:

. Rescaled Range Analysis: A Method for Detecting Persistence, Randomness, or Mean Reversion in Financial Markets

. A method for de-trending asset prices

. Hurst Exponent — Detrended Fluctuation Analysis

[2] In constructing the S&P 500 Fractal Rolling Hurst Regime Detection pipeline, I referenced the following two sources to guide the selection of the *window_size* and *step* parameters:

10 Things You Should Know About Bear Markets

10 Things You Should Know About Bull Markets

window_size = 504 (~2 years of trading days) will captures roughly half to two-thirds of an average bull market (992 days) and nearly 2× the average bear market (289 days). For *step* = 63 (~3 months), it will give 8 overlapping observations per window, providing smooth regime transitions without excessive computation.

[3] For this analysis, I downloaded S&P 500 historical data starting from the year 2000.

Reference:

[1] https://www.nature.com/articles/srep00315

[2] https://en.wikipedia.org/wiki/Detrended_fluctuation_analysis

Further Readings:

[1] Fractal Finance L1: Questionable Research In Finance: How Much Worth Is A Nobel Prize In Economics?

[2] Fractal Finance L2: Modeling Discontinuity And Fractality For Financial Assets

[3] Fractal Finance L3: Modeling Dependency Structures For Financial Assets

Written by Simon Leung

229 followers · 4 following

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.